Test Plan Document

DockWatch: Docker Container Health Monitoring System

Version 1.0 — May 2026 — Systems Engineering Team

Contents

1 Introduction

1.1 Purpose

This document defines the overall testing strategy for DockWatch v1.0. It specifies the scope, approach, resources, and schedule for all testing activities. The goal is to verify that DockWatch meets all functional requirements (FR-01 through FR-06) and selected non-functional requirements defined in the SRS v1.0.

1.2 Scope

Testing covers:

- All six functional requirements (FR-01 to FR-06)
- Key non-functional requirements: NFR-01 (performance), NFR-02 (resource usage), NFR-03 (availability), NFR-04 (security)
- Three testing levels: unit, integration, and system testing
- The web-based dashboard, REST API, WebSocket endpoint, and Docker Engine integration

Out of scope: testing of the Docker Engine itself, host OS configuration, and third-party CI/CD tools.

1.3 References

- DockWatch SRS v1.0, April 15, 2026
- DockWatch Requirement Validation Document v1.0, May 2026
- IEEE Std 829-2008, *IEEE Standard for Software and System Test Documentation*
- Docker Engine API v1.41 Documentation
- OWASP Top 10 Security Risks

1.4 Definitions

Term	Definition
Unit Test	Tests a single function or module in isolation.
Integration Test	Tests the interaction between two or more components.
System Test	End-to-end test of the complete system against requirements.
Pass Criterion	The measurable condition that determines a test has passed.
SUT	System Under Test — the DockWatch application.

2 Testing Strategy

2.1 Testing Levels

DockWatch will be tested at three levels, applied in sequence:

1. **Unit Testing** — Individual backend functions and frontend components tested in isolation. Covers metric calculation logic, YAML parser, JWT validation, and API route handlers.
2. **Integration Testing** — Tests the interaction between the backend and Docker Engine API, between the frontend and REST/WebSocket API, and between the metric collector and the database.
3. **System Testing** — Full end-to-end tests simulating real user scenarios: container failure detection, recovery triggering, dashboard display, and historical data retrieval. Includes performance and security testing.

2.2 Testing Approach

Level	Tools	Responsibility
Unit Testing	pytest (backend), Jest (frontend)	Developers
Integration Testing	pytest, Docker SDK, Postman	QA Engineers
System Testing	Selenium, Locust, OWASP ZAP	QA Engineers
Performance Testing	Locust	QA Engineers
Security Testing	OWASP ZAP, manual review	Security Lead

2.3 Entry and Exit Criteria

Entry Criteria (testing may begin when):

- SRS and Requirement Validation documents are approved.
- Development of the feature under test is complete.
- Test environment (Docker host, database) is configured and accessible.

Exit Criteria (testing phase is complete when):

- All high-priority test cases have been executed.
- No open critical or high-severity defects remain.
- Test coverage $\geq 80\%$ for backend unit tests.
- All system test scenarios pass their defined acceptance criteria.

3 Test Environment

3.1 Hardware

- Linux-based host (x86_64), minimum 2 vCPUs, 4 GB RAM
- Docker Engine installed and running (API v1.41+)

3.2 Software

- DockWatch backend: Python FastAPI
- DockWatch frontend: React (port 3000/3001)
- Database: PostgreSQL (via Alembic migrations)
- Test containers: at least 5 lightweight Docker containers (e.g., nginx, alpine)
- Browser: Chrome (latest) for Selenium tests

3.3 Network

- Backend API: `http://localhost:8000`
- Frontend: `http://localhost:3000`
- WebSocket: `ws://localhost:8000/ws`

4 Unit Test Cases

TC-ID	Test Case	Expected Result	Req.	Priority
UT-01	CPU usage calculation from mock Docker stats	Correct percentage to 2 decimal places	FR-01	High
UT-02	RAM usage calculation from mock stats	Correct MB value returned	FR-01	High
UT-03	Container status parser for running, exited, unhealthy	Correct state string returned	FR-02	High
UT-04	YAML config loader parses valid config	Config object populated correctly	FR-06	High
UT-05	YAML config loader on malformed YAML	Exception raised; no crash	FR-06	Medium
UT-06	JWT validation accepts valid token	Decoded user payload returned	NFR-04	High
UT-07	JWT validation rejects expired token	401 Unauthorized returned	NFR-04	High
UT-08	Recovery rule evaluator triggers restart	Returns action type “restart”	FR-03	High
UT-09	Recovery rule evaluator triggers notify	Returns action type “notify”	FR-03	Medium
UT-10	Telemetry serializer outputs valid JSON	Valid JSON with all required fields	FR-06	Medium

Table 3: Unit Test Cases

5 Integration Test Cases

TC-ID	Test Case	Expected Result	Req.	Priority
IT-01	Backend polls live Docker Engine API	Container list returned with PID, CPU, RAM	FR-01	High
IT-02	REST API `GET /containers`	Status 200; JSON container list	FR-04, FR-06	High
IT-03	REST API `GET /containers/{id}/metrics`	Timestamped metric records returned	FR-04	High
IT-04	WebSocket connection; server pushes update	Client receives update within 5 seconds	FR-05	High
IT-05	Simulated container crash detected by backend	Failure event logged within 30 seconds	FR-02	High
IT-06	Detected failure triggers recovery via Docker SDK	SDK call invoked and action logged	FR-03	High
IT-07	Metrics persisted to database after polling	DB record created with correct values	FR-04	Medium
IT-08	Frontend re-renders on WebSocket update	DOM updates without page reload	FR-05	Medium
IT-09	YAML config change applied without restart	New polling interval takes effect	FR-06	Medium

Table 4: Integration Test Cases

6 System Test Cases

6.1 Functional System Tests

TC-ID	Test Case	Expected Result	Req.	Priority
ST-01	Admin opens dashboard; all containers shown with live metrics	All containers visible; metrics update in real time	FR-01, FR-05	High
ST-02	Admin stops a container; alert appears on dashboard within 30s	Alert visible within 30 seconds	FR-02, FR-05	High
ST-03	System auto-restarts failed container per recovery rule	Container returns to running; event logged	FR-03	High
ST-04	QA queries REST API for 7-day historical CPU data	JSON array of timestamped CPU values returned	FR-04	High
ST-05	Admin updates YAML alert threshold	New threshold applied in next polling cycle	FR-06	Medium
ST-06	10 containers monitored simultaneously	All 10 shown with accurate metrics	FR-01, NFR-05	Medium

Table 5: Functional System Test Cases

6.2 Performance Tests

TC-ID	Test Case	Pass Criterion	Req.	Priority
PT-01	100 concurrent requests to `GET /containers` via Locust	95th percentile response $\leq$ 200ms	NFR-01	High
PT-02	Monitor 50 containers for 10 minutes	Host CPU overhead $<$ 5%	NFR-02	High
PT-03	DockWatch running continuously for 24 hours	No crash; memory stable	NFR-03	Medium

Table 6: Performance Test Cases

6.3 Security Tests

TC-ID	Test Case	Pass Criterion	Req.	Priority
SEC-01	Access API endpoint without JWT token	401 Unauthorized returned	NFR-04	High
SEC-02	Submit expired JWT token	401 Unauthorized returned	NFR-04	High
SEC-03	OWASP ZAP scan against REST API	No high-severity vulnerabilities	NFR-04	High
SEC-04	Verify all traffic uses TLS	No plaintext HTTP connections accepted	NFR-04	High

Table 7: Security Test Cases

7 Requirements Traceability Matrix

Requirement	Description	Test Cases
FR-01	Container discovery and metric polling	UT-01, UT-02, IT-01, ST-01, ST-06
FR-02	Failure detection within 30 seconds	UT-03, IT-05, ST-02
FR-03	Automated recovery actions	UT-08, UT-09, IT-06, ST-03
FR-04	Historical metric storage via REST API	IT-02, IT-03, IT-07, ST-04
FR-05	Real-time dashboard	IT-04, IT-08, ST-01, ST-02
FR-06	YAML configuration and JSON export	UT-04, UT-05, UT-10, IT-09, ST-05
NFR-01	API response time $\leq$ 200ms	PT-01
NFR-02	Host CPU overhead $<$ 5%	PT-02
NFR-03	99.9% uptime	PT-03
NFR-04	JWT authentication and TLS	UT-06, UT-07, SEC-01, SEC-02, SEC-03, SEC-04
NFR-05	50 concurrent containers	ST-06, PT-02

Table 8: Requirements Traceability Matrix

8 Test Schedule and Responsibilities

Phase	Start	End	Owner
Unit Testing	Week 1	Week 2	Developers
Integration Testing	Week 3	Week 4	QA Engineers
System Testing	Week 5	Week 6	QA Engineers
Performance Testing	Week 6	Week 7	QA Engineers
Security Testing	Week 7	Week 8	Security Lead
Defect Fix & Retest	Week 8	Week 9	Developers + QA

Table 9: Test Schedule

9 Defect Management

Defects found during testing will be classified by severity:

- **Critical:** System crash, data loss, security breach — must be fixed before release.
- **High:** Core feature broken, requirement not met — must be fixed before system test sign-off.
- **Medium/Low:** Minor UI issues, non-blocking errors — tracked and fixed in next iteration.

All defects will be logged in the project issue tracker (GitHub Issues) with steps to reproduce, severity, and assigned owner.

10 Sign-Off

Role	Name	Date
Prepared By	Systems Engineering Team	May 2026
Reviewed By	QA Lead	May 2026
Approved By	Project Sponsor	May 2026